UNIVERSITY OF St.Thomas

SEIS 606: Vibe Coding

Fall 2026

Paul Kaefer, Adjunct Professor

Tonight's Agenda

1. Short lecture on testing
2. Working session: breakouts to demo/test each others' apps
3. Debrief/demo to the class
4. Iteration on WordDerby app as a group
5. Walk-through project guidelines

UNIVERSITY OF St.Thomas

Testing 1-2-3...

Looks right isn't the same as is right

AI-generated code *usually* runs, but that doesn't mean it's correct or safe.

We built it fast, so now we verify it.

Tonight: test our own apps.

Forms of testing

Manual/functional: does it do what it should?

Automated: unit tests that re-check things every time you change code.

Usability: can a new person figure it out?

Security/penetration testing: can someone make it do what it shouldn't?

What is penetration testing?

Authorized, deliberate attempts to break into or misuse a system, followed by a report of what was found.

Rules of engagement:

- only test with permission
- stay in scope
- report, don't exploit
- never test apps that aren't yours or a classmate's who agreed

Software teams do this before launch, and often hire outsiders.

The attacker mindset

Anything running in the browser belongs to the user, since they can see it, edit it and rerun it.

Ask "what happens if I do the opposite of what the designer expected?"

The most common failures are boring: exposed secrets, missing checks and trusting input.

Common weak spots

- Secrets (or answers!) exposed in client code
- Unvalidated or unescaped input (injection)
- Broken access control (seeing or changing someone else's data)
- Insecure file handling
- Missing rate limits

Open Worldwide Application Security Project (OWASP) publishes information and tools related to vulnerabilities relating to web applications.



Initial WordDerby analysis in browser

Live demo: right-click, View Source, then open dev tools.

Answer these questions:

1. Is the secret word in the page?
2. Is the word list downloadable?
3. Are there comments, keys or test code left in?

Takeaway: if the game logic lives in the browser, the answer does too.

WordDerby: more advanced pentesting

Suggestions generated by Claude:

- Buttons limit typing, but not tampering. In dev tools you can edit a button's value, re-enable disabled buttons, or call game functions from the console.
- Edit saved state (localStorage) to change scores, lives or streaks.
- Click fast, double-click, or hit keys mid-animation to look for race conditions.
- Refresh mid-game: does it reset, leak the word or let you retry for free?
- Any text anywhere (a player name, a leaderboard)? That's where injection could happen.
- Ordinary bugs count, too: repeated letters, words with hyphens or apostrophes (Uchenna's suggestion last class!), uppercase vs lowercase, win/lose edge cases.

Food4Thought: larger attack surface

- Could a user upload a script named photo.jpg? SVG files can contain scripts—are they allowed?
- Really large files, or thousands of uploads to try and overwhelm our app?
- Filename upload tricks like "../"
- Privacy: do photos keep GPS location data? Can one user open another user's photo by guessing its URL?
- If the app sends photos or text to an AI model, does text inside the image act as a prompt injection ("ignore your instructions…")?
- API keys or database credentials visible in the front-end code or the repo.

Use AI as your red team

Prompt: "Review this code as a security tester. List the top five ways to break or abuse it, ranked by risk, with steps to reproduce."

Then: "Fix issue #1, and explain what changed."

Caution: verify the fixes yourself, because AI reviewers can miss things or invent problems.

Breakout activity

I'll create breakout rooms, groups of 3-4.

Builder demos their app/site.

Testers work through the checklist:

1. use it normally
2. try to break it
3. view source
4. try one security probe from tonight.

Log each finding as what you did (perhaps open issues on GitHub!)

Details: what happened? how bad is the issue?

Bring your findings back to the class.

Debrief/demo

- Let's have 2-3 volunteers demo their app and the most interesting bug/vulnerability found.

- Each group will share at least one takeaway.
  - Students at different places, including how repo was built out
  - Lots of work to be done! Got some done on the fly
  - From a security perspective, things looked good overall (for app using an external LLM, you are leaning on their security)
  - Raphael made a 2D side-scrolling platformer; game worked well, but uncapped velocity & jump timing!

UNIVERSITY OF St.Thomas

Testing & updating WordDerby

Our steps

1. Update the spec. Describe the changes in plain English, or run /speckit-clarify to have Copilot interrogate the spec.
    - Clarify resolves ambiguities in the spec and asks at most three high-impact questions.
2. /speckit-plan to bring the technical approach in line with the new spec.
3. /speckit-tasks to regenerate the task list.
4. /speckit-analyze to check consistency across the spec, plan, and tasks.
    - This only reports and doesn't change anything, so a good chance to review what we've updated.
5. /speckit-implement to execute.

Assignment #4

Your assignment:

Document and (attempt to) address issues flagged during tonight's working session.

Submit report; screenshots encouraged!



Project Requirements

(High-level Overview)

Build an app that works/runs, to some degree.

Structured, document testing.

Analysis & presentation.

UNIVERSITY OF St.Thomas

Questions? Thoughts?